CSE755: Generative Machine Learning

Contents

Probabilistic Modeling, Likelihood, and Latent Variables **2**

Probabilistic Modeling, Likelihood, and Latent Variables

A few reading tips. A superscript in parentheses, as in $\mathbf{x}^{(n)}$, means "the n-th example in the dataset." It is not a power. A subscript, as in x_d or π_k, picks out a coordinate inside one example or one mixture component. A bold letter such as $\mathbf{x}$ or $\mathbf{z}$ usually means a list of numbers (a vector).

Slide 1: Title page

The three keywords in the subtitle are the whole lecture.

- *Probabilistic modeling* means describing data using probabilities.
- *Likelihood* is a score for how well a particular setting of a model explains the data we actually have.
- *Latent variables* are hidden quantities that we never see in the data but that help explain what we do see.

Slide 2: Where this lecture sits

These four letters appear on almost every slide for the rest of the term.

- $\mathbf{x}$: one piece of data, for example one picture. It is bold because it is usually a list of numbers (the pixel values).
- y: a label attached to the data, such as "cat" or "dog."
- $\mathbf{z}$: a hidden variable that is part of the model but never appears in the dataset.
- θ: all the adjustable numbers inside the model (its "knobs"). Training means turning these knobs.

Slide 4: What problem are we solving?

"Generative modeling" definition: given examples

$$\mathcal{D} = \{\mathbf{x}^{(1)}, \ldots, \mathbf{x}^{(N)}\}$$

drawn from an unknown data distribution $p_{\text{data}}(\mathbf{x})$, learn a model $p_\theta(\mathbf{x})$ that (1) assigns high probability to realistic $\mathbf{x}$, called **density estimation**, and (2) can draw new $\mathbf{x} \sim p_\theta$ that were not in $\mathcal{D}$, called **sampling**. Below the box: examples of $\mathbf{x}$ are an image, a sentence, a waveform, a molecule, or a video clip. If labels exist, the same idea extends to the joint $p_\theta(\mathbf{x}, y)$.

What each symbol stands for.

- $\mathcal{D}$: the dataset, the collection of examples we are given.
- $\mathbf{x}^{(n)}$: the n-th example, for example the n-th photo.
- N: how many examples there are.
- $p_{\text{data}}(\mathbf{x})$: the true, unknown "recipe" by which the world produces data. We never get to see it, only samples from it.
- $p_\theta(\mathbf{x})$: our model's guess at that recipe. The subscript θ reminds us that the guess depends on the knobs θ.
- $\mathbf{x} \sim p_\theta$: read "$\mathbf{x}$ is drawn from p_θ."

What it says. Think of p_{data} as a secret recipe book and the dataset as a handful of dishes made from it. We want to build a cook (the model) with two abilities. First, shown any dish, the cook can say how typical it is of the real cuisine (density estimation). Second, the cook can make new dishes that fit the cuisine but are not copies of the ones we tasted (sampling).

Slide 5: Three questions every generative model answers

- **1. Density:** "How probable is this $\mathbf{x}$?" with $p_\theta(\mathbf{x}) = ?$

  Imagine a model that has studied thousands of photos of cats. Now you hand it a new picture, x, and ask: "Does this look like the kind of thing you'd normally see?" A normal cat photo gets a high number.

- **2. Sample:** "How do I draw a new $\mathbf{x}$?" with $\mathbf{x} \sim p_\theta$.
- **3. Represent:** "What hidden factors explain $\mathbf{x}$?" with "$\mathbf{z}$ such that $\mathbf{x} \approx g(\mathbf{z})$."

  Think about describing a cat photo to a friend over the phone. You wouldn't read out every pixel. You'd say: "orange cat, sitting, facing left, green eyes, on a couch." Those few details are enough for your friend to picture something very close to the real photo. Those details are the hidden factors. They're "hidden" because you can't see them written anywhere in the raw pixels. The model has to figure them out.

  - x is the actual example, like the cat photo with its millions of pixel values.
  - z is the short summary of hidden factors (color, pose, lighting, and so on). It's called a latent code, and it's much smaller than x.
  - g is a function that takes the summary z and rebuilds a picture from it.
  - x ≈ g(z) means "if I rebuild the picture from the summary, I get something close to the original." It's "approximately equal," not exact, because a short summary can't hold every tiny detail.

Picture a model of faces. Question 1 asks how face-like a given picture is. Question 2 asks the model to invent a brand-new face. Question 3 asks for a short description (like slider settings for age, smile, and head angle) that could be used to rebuild the face. No single type of model makes all three easy, which is why the course studies several families.

Slide 7: Random variables

A random variable is a rule that assigns a number to each possible result of a random experiment.

For example, when you toss a coin, the result can be Heads or Tails. We can define a random variable X like this:

$$X = \begin{cases} 1, & \text{if the result is Heads,} \\ 0, & \text{if the result is Tails.} \end{cases}$$

Before tossing the coin, we do not know whether X will be 0 or 1.

When rolling a six-sided die, we can define X as the number shown. The possible values are 1, 2, 3, 4, 5, and 6.

There are two types of random variables:

- **Discrete random variable:** Takes separate, countable values. For example, the number of students absent from a class can be 0, 1, 2, 3, and so on.
- **Continuous random variable:** Can take any value within a range, including decimal values. For example, a student's height can be 150 cm, 150.5 cm, or 150.55 cm.

Probability Mass Function (PMF)

A Probability Mass Function (PMF) gives the probability of each specific value of a discrete random variable.

For example, suppose you roll a fair six-sided die. Let X be the number shown. Each number has the same probability:

$$P(X = x) = \frac{1}{6}, \qquad x \in \{1, 2, 3, 4, 5, 6\}.$$

So, the probability of rolling a 3 is $P(X = 3) = \frac{1}{6}$.

Here is another example. Toss a coin twice and let X be the number of Heads. The possible values of X are 0, 1, and 2:

$$P(X = 0) = \frac{1}{4}, \qquad P(X = 1) = \frac{1}{2}, \qquad P(X = 2) = \frac{1}{4}.$$

The probabilities add up to 1. This is true for every PMF.

In simple words, a **PMF gives the probability of each individual value**.

Probability Density Function (PDF)

A Probability Density Function (PDF) is used for a continuous random variable, such as a person's height, the time needed to finish a race, or the temperature of a room.

A continuous variable can take many possible values, including decimal values. Because there are so many possible values, we usually cannot list the probability of every value in a table. Instead, a PDF describes how probability is spread across the possible values.

A PDF is often shown as a curve. The area under the curve over a range gives the probability that the value falls within that range. For example, if X is a student's height, then

$$P(150 \leq X \leq 160)$$

means the probability that the student's height is between 150 cm and 160 cm.

For a continuous random variable, the probability of one exact value is zero in the ideal mathematical model:

$$P(X = 160) = 0.$$

This does not mean a student cannot be measured as 160 cm tall. It means that probability is calculated over a range rather than one infinitely precise value.

The total area under a PDF curve is 1.

Difference Between PMF and PDF

Feature	**PMF**	**PDF**
Used for	Discrete random variables	Continuous random variables
Example	Number shown on a die	Student's height
Probability	Gives probability for each specific value	Area over a range gives probability
Total	All probabilities add up to 1	Total area under the curve is 1

Remember

Use a **PMF** when asking for the probability of an individual, countable result, such as rolling exactly 4 on a die.

Use a **PDF** when working with a continuous quantity, such as the probability that a student's height is between 150 cm and 160 cm.

The main difference is that a PMF gives probabilities for individual values, while a PDF uses area under a curve to find probabilities over ranges.

Slide 8: Joint, marginal, conditional probability

Imagine a class of 20 students. Some wear glasses, some don't, and some are in the school band, some aren't. Here's the full breakdown:

	In band	Not in band	Total
Glasses	4	6	10
No glasses	2	8	10
Total	6	14	20

All three types of probability come from reading this one table in different ways.

Joint probability: "both things at once"

This is the chance that **two things are true at the same time**.

What's the chance a random student wears glasses AND is in the band?

Look at the single cell where both overlap: 4 students out of 20.

$$P(\text{glasses and band}) = \frac{4}{20} = 0.20\ (20\%)$$

Marginal probability: "just one thing, ignore the other"

This is the chance of **one thing alone**, no matter what happens with the other. It's called "marginal" because you find it in the **margins** (the Total row and column) of the table.

What's the chance a random student is in the band?

Ignore the glasses completely and use the band total: 6 out of 20.

$$P(\text{band}) = \frac{6}{20} = 0.30\ (30\%)$$

You get a marginal by adding up the joint probabilities across the other variable:

$$\frac{4+2}{20}$$

Conditional probability: "given that I already know something"

This is the chance of one thing **after you've been told something else is true**. The new information shrinks the group you're looking at.

A student walks in wearing glasses. What's the chance they're in the band?

Now you only care about the 10 students with glasses, because that's the group you know they belong to. Of those 10, 4 are in the band.

$$P(\text{band} \mid \text{glasses}) = \frac{4}{10} = 0.40\ (40\%)$$

The "|" means "given that." Notice this is different from the marginal $P(\text{band}) = 30\%$. Learning about the glasses changed the answer.

How they connect

Conditional probability is a joint divided by a marginal:

$$P(\text{band} \mid \text{glasses}) = \frac{P(\text{glasses and band})}{P(\text{glasses})} = \frac{0.20}{0.50} = 0.40$$

Dividing by $P(\text{glasses})$ is the "shrink the group" step: you're no longer measuring against all 20 students, only the 10 with glasses.

Quick summary

Type	Question it answers	Group you divide by
Joint	Both A and B?	Everyone
Marginal	Just A?	Everyone
Conditional	A, knowing B is true?	Only the B group

One common mix-up to avoid: $P(\text{band} \mid \text{glasses})$ is **not** the same as $P(\text{glasses} \mid \text{band})$. The first asks "of glasses-wearers, how many are in band?" (4/10). The second asks "of band members, how many wear glasses?" (4/6). Different groups, different answers.

The Product Rule

Think of the product rule as the "rearranged" version of conditional probability. It lets you build a joint probability (both things at once) from smaller pieces.

We'll use the same class of 20 students:

	In band	Not in band	Total
Glasses	4	6	10
No glasses	2	8	10
Total	6	14	20

The rule

$$P(A \text{ and } B) = P(A) \times P(B \mid A)$$

In words: *the chance of both things happening = the chance of the first thing × the chance of the second thing, given the first already happened.*

Why it works: step by step

Say we want $P(\text{glasses and band})$. Think of it as two steps:

1. **Pick a student, and hope they wear glasses.**

$$P(\text{glasses}) = \frac{10}{20} = 0.50$$

2. **Now, among only the glasses-wearers, hope this student is in the band.**

$$P(\text{band} \mid \text{glasses}) = \frac{4}{10} = 0.40$$

Multiply the two steps:

$$P(\text{glasses and band}) = 0.50 \times 0.40 = 0.20$$

Check against the table: 4 students out of 20 = 0.20. It matches.

Why multiplying makes sense

Take half of the class (the glasses group), then take 40% of that half. Half of 20 is 10 students, and 40% of 10 is 4 students. Each step shrinks the group, and multiplication does that shrinking.

Where it comes from

It's just the conditional probability formula with both sides multiplied by $P(A)$:

$$P(B \mid A) = \frac{P(A \text{ and } B)}{P(A)} \quad \rightarrow \quad P(A \text{ and } B) = P(A) \times P(B \mid A)$$

Order doesn't matter

You can start with either event:

$$P(A \text{ and } B) = P(B) \times P(A \mid B)$$

Using band first: $P(\text{band}) = 6/20 = 0.30$, and $P(\text{glasses} \mid \text{band}) = 4/6$. Then

$$0.30 \times \frac{4}{6} = 0.20$$

Same answer.

Special case: independent events

If knowing A tells you nothing about B, then

$$P(B \mid A) = P(B)$$

and the rule simplifies to

$$P(A) \times P(B)$$

That's the shortcut you may have seen for things like two coin flips. In our class, $P(\text{band} \mid \text{glasses}) = 0.40$ but $P(\text{band}) = 0.30$, so glasses and band are *not* independent, and we need the full product rule.

Slide 9: Bayes' rule

Bayes' rule is a way to **update what you believe when you get new evidence**.

$$\underbrace{p(y \mid \mathbf{x})}_{\text{posterior}} = \frac{\overbrace{p(\mathbf{x} \mid y)}^{\text{likelihood}} \overbrace{p(y)}^{\text{prior}}}{\underbrace{p(\mathbf{x})}_{\text{evidence}}}.$$

The four named parts.

- **Prior** $p(y)$: your belief about the label before looking at the image.
- **Likelihood** $p(\mathbf{x} \mid y)$: how plausible this image is if the label were y.
- **Posterior** $p(y \mid \mathbf{x})$: your updated belief about the label after seeing the image.
- **Evidence** $p(\mathbf{x}) = \sum_y p(\mathbf{x} \mid y)\, p(y)$: the overall probability of the image, adding up over all labels.

The idea

You start with a guess about how likely something is. Then you see a clue. Bayes' rule tells you how much that clue should change your guess.

An example

Say 1 out of every 100 students in your school has the flu (1%). The school has a test for it, and the test is pretty good: it correctly says "positive" for 90% of students who really have the flu. But it also says "positive" by mistake for 9% of healthy students.

You test positive. What are the chances you actually have the flu? Most people say "about 90%." That's wrong. Let's work it out with 1,000 students:

- 10 students have the flu (1% of 1,000). The test catches 9 of them.
- 990 students are healthy. The test wrongly flags about 89 of them (9% of 990).
- So about 98 students test positive in total (9 + 89).
- Of those 98, only 9 really have the flu.

Chance you have the flu, given a positive test:

$$\frac{9}{98} \approx 9\%$$

The test is good, but the flu is rare, so most positive results come from healthy people getting false alarms.

The formula

Bayes' rule is just that counting written in symbols:

$$P(A \mid B) = \frac{P(B \mid A) \times P(A)}{P(B)}$$

- $P(A \mid B)$: the chance of A, given that B happened (chance of flu, given a positive test). This is what you want.
- $P(B \mid A)$: the chance of B if A is true (chance of a positive test if you have the flu: 90%).
- $P(A)$: how likely A was before any evidence (1% had the flu). This is called the *prior*.
- $P(B)$: how likely B is overall (about 9.8% of everyone tests positive).

Plugging in:

$$\frac{0.90 \times 0.01}{0.098} \approx 0.09, \text{ or } 9\%$$

The takeaway

Evidence matters, but so does what you already knew. A strong clue about a rare thing can still leave the thing unlikely. Bayes' rule is how you combine both properly.

Why 90% Is Wrong

The 90% answer feels right because it mixes up two different questions:

1. **If you have the flu, how likely is a positive test?** This is 90%. It's the test's accuracy.
2. **If you got a positive test, how likely is it that you have the flu?** This is what you actually want to know, and it's about 9%.

They sound almost the same, but they are not. The test's 90% only describes what happens *among sick people*. It says nothing about how many sick people there are in the first place.

Why the second question has a different answer

Think about who ends up in the "positive" group. Two kinds of students are in it:

- Sick students the test correctly caught: **9**
- Healthy students the test wrongly flagged: **about 89**

The healthy group is huge (990 students), so even a small 9% mistake rate produces a lot of false alarms. 9% of 990 is far more than 90% of 10. The false alarms simply outnumber the real catches, roughly 10 to 1.

So if you pick a random student from the positive group, they are much more likely to be a healthy student with a false alarm than a sick student.

An everyday comparison

Imagine a metal detector that beeps for 90% of real knives, but also beeps by accident for 9% of harmless things like belt buckles and keys. If only 1 in 100 people carries a knife, then almost every beep is a belt buckle. A beep is a clue, but not proof, because knives are rare to begin with.

Why intuition fails

Our brains focus on the test's accuracy (the vivid, exciting number) and ignore how rare the flu is (the boring background number). This mistake is called **base rate neglect**. The "base rate" is how common something is before any testing, and it has a big effect on the final answer.

Quick check: if half the school had the flu, the same positive test would be far more convincing. The test hasn't changed, only the base rate has. That is why you can't judge a test result without knowing how common the thing is.

Hand notes: same names, two uses. The four names are reused later with a latent $\mathbf{z}$ in place of the label y:

Name	With a label	With a latent
Prior	$p(y)$	$p(\mathbf{z})$
Likelihood	$p(\mathbf{x} \mid y)$	$p(\mathbf{x} \mid \mathbf{z})$
Joint	$p(\mathbf{x}, y)$	$p(\mathbf{x}, \mathbf{z})$
Posterior	$p(y \mid \mathbf{x})$	$p(\mathbf{z} \mid \mathbf{x})$
Evidence / marginal	$p(\mathbf{x})$	$p(\mathbf{x}) = \int p(\mathbf{x}, \mathbf{z})\, d\mathbf{z}$

Slide 10: Independence and the i.i.d. assumption

This slide has three boxes about when things do or do not affect each other. The first box, **Independence**, is about two things that tell you nothing about each other. The second box, **Conditional independence**, is about two things that stop affecting each other once you know a third thing. The third box, **i.i.d. data**, is about the usual assumption that training examples are separate draws from the same source, which lets us find the chance of a whole dataset by multiplying.

Independence.

$$p(x, y) = p(x)\, p(y) \quad \Longleftrightarrow \quad p(x \mid y) = p(x).$$

Two variables are independent if knowing one tells you nothing about the other. A coin flip and a die roll are independent. The second form says it directly: learning y does not change your belief about x.

Conditional independence.

$$X \perp Y \mid Z \quad \text{means} \quad p(x, y \mid z) = p(x \mid z)\, p(y \mid z).$$

The symbol $\perp$ means "independent of," and $\mid Z$ means "once Z is known." The slide's gloss is that knowing Z makes X irrelevant for predicting Y. Said another way: once you know Z, also learning X adds nothing about Y. Example: two people each carry an umbrella because of the weather. If you know it is raining, one person's umbrella tells you nothing more about the other's. If you do not know the weather, one umbrella makes the other more likely. This idea returns on slide 23.

i.i.d. data.

$$p(\mathcal{D} \mid \theta) = \prod_{n=1}^{N} p\big(\mathbf{x}^{(n)} \mid \theta\big).$$

- i.i.d. stands for *independent and identically distributed*.
- $\prod_{n=1}^{N}$: multiply together the terms for $n = 1, 2, \ldots, N$.
- $p(\mathbf{x}^{(n)} \mid \theta)$: the model's probability for the n-th example when the knobs are θ.

"Identical" means every example comes from the same distribution with one shared θ. "Independent" means the probability of the whole dataset is found by multiplying the probabilities of the individual examples. The equation says: the chance of seeing this whole dataset is the product of the chances of each example.

An important subtlety. The i.i.d. assumption is about *different training examples* (different images). It is *not* true of pixels inside one image. A blue pixel in a sky tells you its neighbor is probably blue too. Capturing that dependence inside each example is exactly the job of a generative model, as the slide says.

Slide 11: Likelihood is a function of the parameters

This slide shows that one formula, $p(\text{data} \mid \theta)$, can be read in two different ways. In the first box, we keep the knobs θ fixed and ask what data we might see. In the second box, we keep the data fixed and ask which knob settings explain it best. Under the boxes are two equations, one for the likelihood and one for the log-likelihood, and a single sentence about why we take the log.

- **Probability box:** θ fixed, $\mathbf{x}$ varies. $p(\mathbf{x} \mid \theta)$ is a distribution over data.
- **Likelihood box:** $\mathbf{x}$ fixed (the dataset), θ varies. $L(\theta) = p(\mathcal{D} \mid \theta)$. L is *not* a probability distribution over θ unless we add a prior.

The key idea. It is the *same formula*, $p(\text{data} \mid \theta)$, read two ways.

- As a **probability**, the knobs are fixed and we ask what data we might see. Take a coin known to be fair: what flips are likely?
- As a **likelihood**, the data are fixed and we ask which knob settings explain it well. We already saw the flips: which coin bias is most believable?

Because the likelihood is a function of θ, its values at different θ do not have to add up to 1. That is why it is not a probability distribution over θ. To turn it into one you need a prior over θ and Bayes' rule (hand notes).

The equations.

$$L(\theta) = \prod_{n=1}^{N} p\big(\mathbf{x}^{(n)} \mid \theta\big), \qquad \ell(\theta) = \log L(\theta) = \sum_{n=1}^{N} \log p\big(\mathbf{x}^{(n)} \mid \theta\big).$$

- $L(\theta)$: the likelihood. The first formula is the i.i.d. product from slide 10.
- $\ell(\theta)$: the **log-likelihood** (a curly "l").
- log: the logarithm, which turns multiplication into addition: $\log(ab) = \log a + \log b$.

The equations say: the log-likelihood is just the likelihood after taking a log, and thanks to $\log(ab) = \log a + \log b$ the product turns into a sum.

Why maximize ℓ instead of L? The slide gives two reasons.

1. *Products of probabilities underflow.* Multiplying many numbers smaller than 1 gives a tiny result. For instance, 1000 numbers each equal to 0.01 multiply to 10^{-2000}, which is smaller than the

smallest number a computer stores, so it becomes 0. The sum of the (natural) logs is about -4605, which is no problem.

2. *The log is strictly increasing.* A bigger L always gives a bigger $\log L$, so the θ that maximizes one maximizes the other.

Slide 12: Maximum likelihood estimation

This slide explains maximum likelihood estimation (MLE), which simply means choosing the knob settings that make the data we saw as likely as possible. A blue box holds the main formula. Below it, an equation shows why this goal makes sense: it is the same as making the model as close as possible to the real data. A sentence about entropy follows. A red box at the bottom warns that the direction of the KL divergence matters.

The MLE formula.

$$\hat{\theta}_{\mathrm{ML}} = \arg\max_{\theta} \ \ell(\theta) = \arg\max_{\theta} \sum_{n=1}^{N} \log p\big(\mathbf{x}^{(n)} \mid \theta\big).$$

- $\hat{\theta}_{\mathrm{ML}}$: the maximum-likelihood estimate. The hat means "our estimate of," and ML stands for maximum likelihood.
- $\arg\max_{\theta} f(\theta)$: "the value of θ that makes f biggest." It returns the best θ, not the best score.

This says: pick the knob settings that make the data we saw as probable as possible.

Why this is the right goal. The slide states

$$\mathbb{E}_{p_{\mathrm{data}}}\big[\log p_\theta(\mathbf{x})\big] = -H(p_{\mathrm{data}}) - \mathrm{KL}\big(p_{\mathrm{data}} \,\|\, p_\theta\big).$$

- $\mathbb{E}_{p_{\mathrm{data}}}[\cdots]$: the average of the quantity inside when $\mathbf{x}$ is drawn from the true distribution. In words, how well the model scores on a fresh data point, on average.
- $H(p_{\mathrm{data}})$: the **entropy** of the true distribution, $H(p_{\mathrm{data}}) = -\int p_{\mathrm{data}} \log p_{\mathrm{data}}$. It measures how spread out or unpredictable the data are. It depends only on nature, not on θ.
- $\mathrm{KL}(p \,\|\, q) = \mathbb{E}_p[\log p(\mathbf{x}) - \log q(\mathbf{x})]$: the **Kullback–Leibler divergence**, a measure of how different q is from p. It is never negative, and it equals 0 exactly when $p = q$ (almost everywhere). It is distance-like but not symmetric: swapping p and q changes the answer.

What the equation says: the model's average score equals a fixed number set by nature, minus how far the model is from reality. Only the second part depends on θ. So making the score as high as possible is the same as making the KL divergence as small as possible. That is the precise meaning of "choose θ so that p_θ is close to p_{data}" from slide 6. The sum over the dataset in the MLE formula is a stand-in for this average, since we can only average over the examples we actually have. (Getting from the definition of KL to this identity is homework Problem 3.2, so the algebra is left for you.)

Direction matters. $\mathrm{KL}(p_{\mathrm{data}} \| p_\theta)$ is large when the model puts almost no probability where real data live. So a model trained this way tries hard not to miss any cluster of data. This is called **mode covering**. The reverse, $\mathrm{KL}(p_\theta \| p_{\mathrm{data}})$, is large when the model puts probability where there is no data. It is used by some variational methods and tends to be **mode seeking**: the model may lock onto one cluster and ignore the rest. Picture data in two separate clumps. A mode-covering model spreads over both clumps. A mode-seeking model may sit on just one (hand notes).

Slide 13: Worked example: Bernoulli MLE

This slide walks through a complete MLE example using coin flips. The model has just one knob, the chance of heads. The slide writes down the score for the flips we saw, uses a little calculus to find the knob setting with the best score, and discovers that the answer is simply the fraction of flips that came

up heads. A green "Lesson" box at the end explains what changes when we move from coins to deep models.

Setup. Each flip $x^{(n)}$ is 1 (say, heads) or 0 (tails). The model has one knob: $p(x = 1 \mid \theta) = \theta$, the chance of heads. Let N_1 be the number of heads, N_0 the number of tails, and $N = N_1 + N_0$.

Log-likelihood.

$$\ell(\theta) = \sum_{n=1}^{N} \Big[x^{(n)} \log \theta + \big(1 - x^{(n)}\big) \log(1 - \theta) \Big].$$

- For a flip with $x^{(n)} = 1$, the second term vanishes (it is multiplied by 0), leaving $\log \theta$.
- For a flip with $x^{(n)} = 0$, the first term vanishes, leaving $\log(1 - \theta)$.

The bracket is a switch: it picks the right log-probability for whichever outcome happened. Adding over all flips and grouping heads and tails gives the hand-notes form

$$\ell(\theta) = N_1 \log \theta + N_0 \log(1 - \theta).$$

Take the derivative and set it to zero.

$$\frac{\partial \ell}{\partial \theta} = \frac{N_1}{\theta} - \frac{N_0}{1 - \theta} = 0 \quad \Longrightarrow \quad \hat{\theta} = \frac{N_1}{N}.$$

- $\partial \ell / \partial \theta$: the slope of ℓ as θ changes. At the top of a hill the slope is zero.

Step by step: $N_1/\theta = N_0/(1 - \theta)$ gives $N_1(1 - \theta) = N_0\theta$, so $N_1 = (N_0 + N_1)\theta = N\theta$, so $\theta = N_1/N$. The second derivative, $-N_1/\theta^2 - N_0/(1 - \theta)^2$, is negative for every θ between 0 and 1, which confirms this is a peak and not a valley (hand notes).

What it says. The best coin bias is simply the fraction of flips that came up heads.

Lesson. For this simple model the MLE is the empirical frequency, the thing you would have guessed. For deep generative models the principle is identical, but $\log p_\theta(\mathbf{x})$ has no tidy formula. So we maximize it by gradient descent (walking uphill step by step), or replace it with a bound, or give up on it. That fork is the plot of the rest of the course.

Slide 14: What "a good density" buys you

This slide lists four useful things you can do once your model gives a good score for how likely any piece of data is. You can **compress** data into fewer bits. You can spot **unusual inputs** (anomaly detection). You can **fill in or make up** missing parts (sampling and imputation). And you can build a **classifier** out of the model using Bayes' rule (downstream Bayes).

Compression and bits per dimension.

$$\text{bpd}(\mathbf{x}) = \frac{-\log_2 p_\theta(\mathbf{x})}{D}.$$

- $-\log_2 p_\theta(\mathbf{x})$: roughly how many **bits** (0/1 digits) are needed to write down $\mathbf{x}$ using a code built from the model. A rare thing has small probability and so needs a long code. A common thing needs a short one, as in Morse code, where the common letter E is a single dot.
- D: the number of coordinates in $\mathbf{x}$. The slide says pixels (or channels times pixels). The hand notes say $H \cdot W \cdot 3$ for an RGB image of height H and width W.
- bpd: bits per dimension, the cost divided by D so images of different sizes can be compared.

The equation says: the average number of bits the model spends per number in the image. A lower value means better compression, so a better density model. As a yardstick (hand notes), a flat distribution over the 256 values $\{0, \ldots, 255\}$ costs $\log_2 256 = 8$ bits per coordinate, and a useful model must do better than that.

Anomaly detection. A very small $p_\theta(\mathbf{x})$ flags an unusual input. The slide warns to use this with care, because in high dimensions likelihoods can rank outliers badly. The hand notes say to treat it as a score, not a proof.

Sampling and imputation. A joint distribution lets you condition on the part you know to fill in the part you do not, such as missing pixels or words.

Downstream Bayes. A class-conditional density $p(\mathbf{x} \mid y)$ plus a prior $p(y)$ gives a classifier through Bayes' rule (slide 18).

Slide 15: Two ways to factor the joint

This slide starts with one equation that breaks the joint probability of a picture and its label into pieces in two different ways. The first way, called the discriminative path, starts from the picture and works toward the label. The second way, called the generative path, starts from the label and works toward the picture. Two boxes under the equation describe the kind of model that goes with each path. The hand notes put it like this: start from the product rule and stop at different factors.

$$p(\mathbf{x}, y) = \underbrace{p(y \mid \mathbf{x})\, p(\mathbf{x})}_{\text{discriminative path}} = \underbrace{p(\mathbf{x} \mid y)\, p(y)}_{\text{generative path}}.$$

This is the product rule from slide 8, now used to define two kinds of models.

Discriminative model. It learns $p(y \mid \mathbf{x})$, or a decision score $f(\mathbf{x})$. Examples on the slide: logistic regression, ResNet classifiers, and most BERT fine-tunes for classification. The slide's key point is that the model never defines how images are produced.

Generative model. It learns $p(\mathbf{x})$, or $p(\mathbf{x} \mid y)$ together with $p(y)$, or the full joint $p(\mathbf{x}, y)$. Examples on the slide: Naive Bayes, PixelCNN, VAE, diffusion, and language models $p(\text{text})$. It can *simulate* new data.

What the symbols stand for.

- $p(y \mid \mathbf{x})$: the chance of each label given the input. This is what a classifier outputs.
- $f(\mathbf{x})$: a decision score, a number computed from $\mathbf{x}$ whose size tells you which class to pick. It need not be a true probability.
- $p(\mathbf{x})$: how likely the input itself is. A discriminative model ignores this factor.

Plain English. A discriminative model is like a judge who looks at a picture and says "cat" or "dog." The judge may be excellent, but cannot draw a cat. A generative model is like an artist who understands what cats and dogs look like well enough to draw new ones, and can also be used to judge.

Slide 16: Diagram: what each model is allowed to do

This slide has two drawings, each made of circles and boxes joined by arrows. The top drawing is the discriminative model: a picture $\mathbf{x}$ goes into a box $p(y \mid \mathbf{x})$, and a label y comes out. The bottom drawing is the generative model: it starts with a label y (chosen using the prior $p(y)$), passes through a box $p(\mathbf{x} \mid y)$, and ends with a picture $\mathbf{x}$. The caption says the arrows show the direction of sampling. So the top model reads a picture and gives a label, while the bottom model can draw a label and then make a picture. If there are no labels at all, you drop y and just draw $\mathbf{x} \sim p(\mathbf{x})$.

- **Discriminative** (top): a blue circle $\mathbf{x}$, then an arrow into a box $p(y \mid \mathbf{x})$, then an arrow into a reddish circle y.
- **Generative** (bottom): a reddish circle y with "prior $p(y)$" written under it, then an arrow into a box $p(\mathbf{x} \mid y)$, then an arrow into a blue circle $\mathbf{x}$.

Plain English. Notice that the arrows point in opposite directions. The top chain goes from picture to label, so you can only hand it a picture and get a label. The bottom chain starts from a label (chosen

using the prior $p(y)$) and ends at a picture, so it can *manufacture* pictures. The reddish circle is the label and the bluish circle is the image in both rows, so the only thing that changes is the direction of the arrow. If there are no labels at all, the generative chain shrinks to a single step that draws $\mathbf{x}$ directly.

Slide 17: Comparison

This slide has a table that compares discriminative and generative models row by row: what each one learns, the typical training score, whether you can sample new data, how each handles missing data, how much data each needs, and how each tends to fail. Below the table is a paragraph about a 2001 paper by Ng and Jordan, which compared the two kinds of models when you have little data and when you have a lot.

	Discriminative	Generative
Object	$p(y \mid \mathbf{x})$ or $f(\mathbf{x})$	$p(\mathbf{x})$ or $p(\mathbf{x}, y)$
Typical loss	cross-entropy on labels	log-likelihood, ELBO, adversarial, score
Can sample $\mathbf{x}$?	No	Yes (quality varies)
Missing data	Hard	Condition the joint
Data efficiency	Often better for pure prediction	Needs to explain all of $\mathbf{x}$
Failure mode	Overconfident OOD predictions	Mode collapse, blur, high cost

Reading the rows.

- *Object:* what is being learned (slide 15).
- *Typical loss:* the training score. Cross-entropy on labels measures how well predicted labels match true labels. The generative list names log-likelihood (slides 11–12), the ELBO (a computable lower bound on log-likelihood, Week 3), an adversarial loss (GANs, Week 4), and a score-based loss (used with diffusion, later weeks).
- *Missing data:* a generative model that holds the joint can fill in gaps by conditioning. A pure classifier needs a separate model for that (hand notes).
- *Data efficiency:* a discriminative model only has to explain the label. A generative model has to explain *all* of $\mathbf{x}$, which is a harder job.
- *Failure mode:* OOD means out-of-distribution, inputs unlike the training data. A classifier can be confidently wrong on such inputs. Generative models can suffer mode collapse (producing only a few kinds of output), blurry samples, or high computational cost.

Ng and Jordan (2001). As the number of training examples N grows without limit ($N \to \infty$), discriminative classifiers can reach a lower prediction error. A generative classifier often reaches its own (possibly higher) final error level with fewer labeled examples, because it also uses the structure of $p(\mathbf{x} \mid y)$ (hand notes). In plain words: with little labeled data the generative approach may get decent sooner, while with lots of data the discriminative approach tends to win at pure classification.

Takeaway. The slide ends by saying to use a generative model when you need samples, a density, or a representation, not only a label. The hand notes sharpen this: for this course the reason to be generative is *not* a small-label classifier. It is samples, density, and representation.

Slide 18: Generative models that still classify

This slide shows that a generative model can also be used to sort things into classes. One equation says to pick the label with the biggest value of likelihood times prior. A teal box introduces Gaussian discriminant analysis, an older method that models each class as its own bell-shaped cloud of points. Two bullets at the end explain how this old idea can be upgraded into the deep models we will study.

Classify using Bayes.

$$\hat{y} = \arg\max_{y} \; p(\mathbf{x} \mid y)\, p(y).$$

- $\hat{y}$: the predicted label.
- $\arg\max_y$: the label y that makes the expression after it biggest.
- $p(\mathbf{x} \mid y)\, p(y)$: likelihood times prior, the top of Bayes' rule on slide 9.

This says: for a new image, compute a score for every label and pick the label with the biggest score. The denominator $p(\mathbf{x})$ of Bayes' rule is the same for every label, so it cannot change which label wins and can be dropped.

Gaussian discriminant analysis.

$$p(y) = \pi_y, \qquad p(\mathbf{x} \mid y) = \mathcal{N}(\mathbf{x}; \boldsymbol{\mu}_y, \boldsymbol{\Sigma}_y).$$

- π_y: the prior probability of class y, that is, how common the class is.
- $\mathcal{N}(\mathbf{x}; \boldsymbol{\mu}, \boldsymbol{\Sigma})$: the Gaussian (bell-curve) density evaluated at $\mathbf{x}$.
- $\boldsymbol{\mu}_y$: the center of class y's bell.
- $\boldsymbol{\Sigma}_y$: the covariance matrix, which describes how spread out the bell is and which directions it stretches in.

This says: each class is modeled as its own bell-shaped cloud of points, and each class has a prior probability.

Which boundary you get. If all classes share one covariance, the dividing line between two classes is straight, and the method is called **LDA** (linear discriminant analysis). If each class has its own covariance, the boundary is curved, and the method is **QDA** (quadratic discriminant analysis). (The reason, a standard fact, is that the squared terms cancel out when the covariances are equal.)

Why this is on a generative-models slide. The slide calls this the classical template that deep models generalize. You can (1) replace the Gaussian with a neural density such as an autoregressive network, a flow, or diffusion, and (2) replace the discrete label y with a continuous latent $\mathbf{z}$. The second step leads straight into the next slides.

Slide 19: Why introduce variables we never observe?

This slide asks a fair question: why would we add variables to a model if we never see them in the data? A box defines a latent (hidden) variable as a hidden, compressed cause of the data, with examples such as which digit it is, its pose, an object's color, or the topic of a document. The slide then gives three reasons why hidden variables are worth having.

Latent variable. A latent (hidden) variable $\mathbf{z}$ is part of the model but is not in the dataset. It is a compressed *cause* of $\mathbf{x}$. The slide's examples are digit identity, pose, object color, and a topic in a document.

The three reasons, in plain English.

1. *The marginal $p(\mathbf{x})$ is complicated, but $p(\mathbf{x} \mid \mathbf{z})$ is simple.* The overall pattern of the data can be messy, but once you know the hidden cause, what is left is easy to describe, for example one bell curve or a decoder network. As an analogy, the heights of all people in a room form a lumpy shape, but the heights within one group (say, one age group) look like a single clean bell.

2. $\mathbf{z}$ *is a usable representation.* The hidden code can be used for editing, clustering, or other tasks.
3. *Dependencies are explained by a shared cause.* Pixels in an image depend on each other, but given $\mathbf{z}$ they may be independent. This is developed on slide 23.

Slide 20: The generative story

This slide tells the story of how a model with a hidden variable makes data. A small picture shows the knobs θ pointing to a hidden cause $\mathbf{z}$ (with its prior $p_\theta(\mathbf{z})$), and then an arrow from $\mathbf{z}$ to the observed data $\mathbf{x}$, labeled $p_\theta(\mathbf{x} \mid \mathbf{z})$. Next comes a two-step recipe for making new data: first pick the hidden cause, then make the observation from it. Last, the slide gives the formula for the marginal likelihood, which is what we use for learning, since we never get to see $\mathbf{z}$. In the picture, $\mathbf{z}$ is the reddish circle and $\mathbf{x}$ is the bluish circle.

Sampling ("ancestral," along the arrows).

1. Draw $\mathbf{z} \sim p_\theta(\mathbf{z})$. The slide's gloss: "choose the hidden cause."
2. Draw $\mathbf{x} \sim p_\theta(\mathbf{x} \mid \mathbf{z})$. The gloss: "render the observation."

Ancestral sampling just means following the arrows: draw the "parents" first, then draw each "child" given its parents.

Learning never sees z. Because $\mathbf{z}$ is not in the dataset, we cannot train on it directly. We maximize the **marginal likelihood** instead:

$$p_\theta(\mathbf{x}) = \int p_\theta(\mathbf{x} \mid \mathbf{z})\, p_\theta(\mathbf{z})\, d\mathbf{z} \qquad \text{or} \qquad \sum_{\mathbf{z}} p_\theta(\mathbf{x} \mid \mathbf{z})\, p_\theta(\mathbf{z}).$$

- $p_\theta(\mathbf{z})$: the prior, how likely each hidden cause is.
- $p_\theta(\mathbf{x} \mid \mathbf{z})$: how likely the observation is, given that cause.
- $\int \cdots d\mathbf{z}$ (continuous $\mathbf{z}$) or $\sum_{\mathbf{z}}$ (discrete $\mathbf{z}$): add over every possible hidden cause.

What the equation says: we do not know which hidden cause produced $\mathbf{x}$, so we try every cause. For each one we multiply "how likely is this cause" by "how likely is this $\mathbf{x}$ under that cause," and then add everything up. This is just the sum rule and product rule from slide 8 applied with $\mathbf{z}$ in place of y.

Hand notes warning. Do not write $p(\mathbf{x}) = \int p(\mathbf{x} \mid \mathbf{z})\, d\mathbf{z}$ and forget the prior $p(\mathbf{z})$. That is one of the listed point-losing mistakes.

Slide 21: Marginal likelihood, posterior, and the hard integral

This slide describes three related probabilities that all come from one model with a hidden variable: the joint, the marginal likelihood, and the posterior. A red box then explains the big problem. The marginal likelihood is very hard to compute when the hidden variable is continuous and the decoder is a neural network. The box ends by pointing to the weeks of the course that each take a different path around this problem.

Three distributions, one model.

$$p_\theta(\mathbf{x}, \mathbf{z}) = p_\theta(\mathbf{x} \mid \mathbf{z})\, p_\theta(\mathbf{z}), \qquad p_\theta(\mathbf{x}) = \int p_\theta(\mathbf{x}, \mathbf{z})\, d\mathbf{z}, \qquad p_\theta(\mathbf{z} \mid \mathbf{x}) = \frac{p_\theta(\mathbf{x}, \mathbf{z})}{p_\theta(\mathbf{x})}.$$

- **Joint** $p_\theta(\mathbf{x}, \mathbf{z})$: the chance of a particular cause *and* a particular observation together.
- **Marginal likelihood** (also called the evidence) $p_\theta(\mathbf{x})$: the chance of the observation with the cause added away.
- **Posterior** $p_\theta(\mathbf{z} \mid \mathbf{x})$: given that we saw $\mathbf{x}$, how likely each cause is. The hand notes call it the distribution over explanations of a particular $\mathbf{x}$.

Intractability. If $\mathbf{z}$ is continuous and $p(\mathbf{x} \mid \mathbf{z})$ is a neural network, the integral that defines $p(\mathbf{x})$ has no closed form, meaning there is no formula that gives the answer. The posterior is hard for the same reason. Look at the posterior formula: the top, $p_\theta(\mathbf{x}, \mathbf{z})$, is easy (just multiply the prior by the decoder). The bottom, $p_\theta(\mathbf{x})$, is the hard integral.

"This single fact splits the rest of the course." Each response is a different way around the hard integral:

Response	Where
Avoid latents. Factor $p(\mathbf{x})$ with the chain rule so every term is an explicit conditional (exact likelihood).	Week 2, autoregressive models
Keep latents, and optimize the ELBO instead of $\log p(\mathbf{x})$.	Week 3, VAEs
Make the map from noise to $\mathbf{x}$ invertible, so the density is a base density times a Jacobian factor (the slide says the integral becomes a determinant).	Week 8, flows
Define a sampler and do not compute $p(\mathbf{x})$ (hand notes only; not on this slide).	Week 4, GANs

ELBO stands for evidence lower bound: a number we *can* compute that is guaranteed to be at most $\log p(\mathbf{x})$, so pushing it up pushes the real thing up too. Details come in Week 3.

Slide 22: Plate notation

This slide teaches plate notation, a shortcut for drawing a model that has many data points. In the picture, a dashed rectangle (the **plate**) holds a hidden circle $\mathbf{z}_n$ sitting above an observed circle $\mathbf{x}_n$, with an arrow pointing from the top one to the bottom one. A letter N in the corner of the rectangle means "repeat this N times." Outside the rectangle is a small square θ (the knobs) with arrows into both circles, because every data point shares the same knobs. Three bullets explain how to read the picture, and an equation gives the probability of the whole dataset.

Reading the diagram.

- The slide says a shaded node is observed and an open node is latent. In the actual picture both circles carry a light tint (warm for $\mathbf{z}_n$, cool for $\mathbf{x}_n$), so the rule is told to you in words more than shown by shading. Just remember: $\mathbf{x}$ is observed, $\mathbf{z}$ is hidden.
- The plate means "repeat everything inside for $n = 1, \ldots, N$." So the diagram stands for N copies of the pair $(\mathbf{z}_n, \mathbf{x}_n)$, one per data point.
- θ sits *outside* the plate, because there is one model whose knobs are shared by every data point.

Each $\mathbf{z}_n$ explains only its own $\mathbf{x}_n$. Given θ, the data points are independent, which restores the product form of the likelihood:

$$p(\mathcal{D} \mid \theta) = \prod_{n=1}^{N} \int p(\mathbf{x}_n \mid \mathbf{z}_n, \theta)\, p(\mathbf{z}_n \mid \theta)\, d\mathbf{z}_n.$$

- $\prod_{n=1}^{N}$: multiply over all N data points (the i.i.d. product from slide 10).
- $\int \cdots d\mathbf{z}_n$: for each data point, add over all its possible hidden causes (the marginal likelihood from slide 20).
- $p(\mathbf{x}_n \mid \mathbf{z}_n, \theta)$ and $p(\mathbf{z}_n \mid \theta)$: the decoder and the prior, now written with θ shown explicitly.

What the equation says: the chance of the whole dataset is the product, over data points, of each point's marginal likelihood with its own hidden cause summed out.

Notation warning. On this slide the data point index is written as a subscript ($\mathbf{x}_n$). Elsewhere in the lecture, in the hand notes, and in the homework, it is a superscript $\mathbf{x}^{(n)}$. They mean the same thing.

The hand notes say not to swap the two conventions in your own derivations, so use the superscript form.

Slide 23: Conditional independence given the latent

This slide shows a hidden variable $\mathbf{z}$ at the top, with arrows going down to many observed pieces x_1, x_2, and so on up to x_D (for example, the pixels of an image). The formula says that once you know $\mathbf{z}$, the pieces no longer affect each other, so the chance of the whole $\mathbf{x}$ is found by multiplying the chances of the pieces.

The assumption.

$$p(\mathbf{x} \mid \mathbf{z}) = \prod_{d=1}^{D} p(x_d \mid \mathbf{z}).$$

- x_d: the d-th coordinate of $\mathbf{x}$ (for example, the d-th pixel).
- D: how many coordinates there are.
- $\prod_{d=1}^{D}$: multiply over all coordinates.

This says: once you know the hidden cause $\mathbf{z}$, the coordinates of $\mathbf{x}$ no longer influence each other, so the chance of the whole $\mathbf{x}$ is the product of the chances of each coordinate. That is the conditional independence from slide 10, with $\mathbf{z}$ playing the role of "what is known."

Unconditionally they still depend on each other. The slide stresses that x_1 and x_2 are dependent *through* $\mathbf{z}$. Integrating (or summing) $\mathbf{z}$ out destroys the product.

A numeric illustration. Let z be the weather: rain or sun, each with probability 0.5. Let x_1 and x_2 say whether two different people carry umbrellas. Given rain, each carries one with probability 0.9, independently. Given sun, each carries one with probability 0.1, independently. Then $p(x_1 = 1) = 0.5 \cdot 0.9 + 0.5 \cdot 0.1 = 0.5$, and likewise $p(x_2 = 1) = 0.5$, so $p(x_1 = 1)\, p(x_2 = 1) = 0.25$. But the chance both carry umbrellas is $0.5 \cdot 0.9^2 + 0.5 \cdot 0.1^2 = 0.405 + 0.005 = 0.41$. Since $0.41 \neq 0.25$, the two are *not* independent, even though they are independent once the weather is known. Seeing one umbrella makes rain more likely, and that makes the other umbrella more likely.

Why it matters. This is how a low-dimensional hidden cause creates a high-dimensional, correlated observation. The slide lists mixtures, factor analysis, VAEs, and many diffusion conditioners as using a version of this idea. The hand notes warn against the logical slip of concluding $x_1 \perp x_2$ from $x_1 \perp x_2 \mid z$.

Slide 24: Running example: a Gaussian mixture

This slide introduces the Gaussian mixture, the main example we will use for the rest of the lecture. A hidden variable z picks which of K clusters a data point belongs to, and each cluster is shaped like a bell curve. The slide gives formulas for the model, the overall density, and the *responsibility* (how likely each cluster is for a given point), along with four bullets and a small plot. The plot shows two separate bell curves on a line labeled x: a teal one on the left ($k = 1$) and an orange one on the right ($k = 2$), which only just touch at their feet. The caption says the overall curve is the sum of the two, not a single Gaussian, but note that the plot draws only the two separate curves, not their sum.

The model. A discrete latent $z \in \{1, \ldots, K\}$ says which cluster, and the observation $\mathbf{x} \in \mathbb{R}^d$ is continuous.

$$p(z = k) = \pi_k, \qquad p(\mathbf{x} \mid z = k) = \mathcal{N}(\mathbf{x}; \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k).$$

$$p(\mathbf{x}) = \sum_{k=1}^{K} \pi_k \, \mathcal{N}(\mathbf{x}; \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k).$$

- K: the number of clusters (components).

- π_k: the probability of picking cluster k. These are the **mixing weights**, with $\pi_k \geq 0$ and $\sum_k \pi_k = 1$.
- $\boldsymbol{\mu}_k$: the center of cluster k.
- $\boldsymbol{\Sigma}_k$: the spread and shape of cluster k.
- $\mathbb{R}^d$: lists of d real numbers. (This slide uses lowercase d; slide 23 used D.)

What it says. To generate a point: first pick a cluster k with probability π_k, then draw the point from that cluster's bell curve. The overall density of $\mathbf{x}$ is a *weighted sum* of the K bell curves, because $\mathbf{x}$ could have come from any cluster. The sum is easy to compute because K is small. That makes this model a rare case where the marginal likelihood from slide 20 is tractable.

The four bullets map onto the Bayes names: the prior is the categorical weights $\boldsymbol{\pi}$, the likelihood is one Gaussian per component, the marginal is the weighted sum, and the posterior is the responsibility.

Responsibility.

$$r_k(\mathbf{x}) = p(z = k \mid \mathbf{x}) = \frac{\pi_k \mathcal{N}_k(\mathbf{x})}{\sum_j \pi_j \mathcal{N}_j(\mathbf{x})}.$$

- $r_k(\mathbf{x})$: the **responsibility** of cluster k for the point $\mathbf{x}$, meaning the posterior probability that $\mathbf{x}$ came from cluster k.
- $\mathcal{N}_k(\mathbf{x})$: shorthand for $\mathcal{N}(\mathbf{x}; \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$.
- $\sum_j$: add over all clusters, using j as the counter so it is not confused with the specific k on top.

This is Bayes' rule (slide 9) with the cluster label as the unknown. The top is prior times likelihood for cluster k, and the bottom adds that quantity over all clusters, which is the evidence. The result is a number between 0 and 1 for each cluster, and the numbers over all clusters add to 1. It is a *soft* assignment: a point between two clusters can be, say, mostly cluster 1 and partly cluster 2.

Slide 25: Why EM appears (preview)

This slide gives a preview of a method called Expectation–Maximization, or EM. It begins with the log-likelihood of the mixture model, which is hard to maximize because it has a log sitting outside a sum. A blue box then describes EM as a simple two-step routine that guesses which cluster each point belongs to and then refits the clusters, over and over. A closing paragraph connects EM to the VAE we will meet in Week 3.

The difficulty.

$$\ell(\theta) = \sum_{n=1}^{N} \log \sum_{k=1}^{K} \pi_k \mathcal{N}\big(\mathbf{x}^{(n)}; \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k\big).$$

This is the log-likelihood from slide 11 with the mixture density from slide 24 plugged in. The trouble is that it has a *log of a sum*. The log cannot pass through a sum: $\log(a + b)$ is not $\log a + \log b$. So the parameters stay tangled together, and setting derivatives to zero does not give a simple linear system with a neat answer. The hand notes list the mix-up as a mistake: $\log \sum_k \pi_k \mathcal{N}_k$ is *not* $\sum_k \log(\pi_k \mathcal{N}_k)$.

Expectation–Maximization (EM). It repeats two steps.

- **E-step.** Freeze θ and compute the responsibilities $r_k(\mathbf{x}^{(n)})$ for every point and cluster, a soft assignment of each point to the clusters.
- **M-step.** Freeze those responsibilities and update π_k, $\boldsymbol{\mu}_k$, $\boldsymbol{\Sigma}_k$ in closed form, as if the assignments had been observed.

Plain English. It is a chicken-and-egg problem. If you knew which cluster each point belonged to, finding the cluster centers would be easy. If you knew the centers, finding which cluster each point belongs to would be easy. EM alternates: guess the assignments (softly), then re-fit the clusters to those guesses, and repeat.

The slide says EM maximizes the marginal likelihood by repeatedly lower-bounding it. The hand notes add that each EM iteration does not decrease the marginal likelihood. The VAE (Week 3) is the same idea with two changes. A neural *recognition network* plays the role of the E-step, and an ordinary gradient step plays the role of the M-step. The bound itself is derived in Week 3.

Exam guidance from the hand notes. Do not memorize the EM update formulas beyond the responsibility and the statement of the two steps, unless a problem asks you to derive a specific M-step.

Slide 26: Complete data vs. incomplete data

This slide has two boxes side by side. The teal box, **Complete**, shows the easy case where we get to see both the data $\mathbf{x}$ and the hidden variable $\mathbf{z}$. The red box, **Incomplete**, shows the real case where we only see $\mathbf{x}$, so the log ends up outside an integral. The slide then brings in Jensen's inequality, a math trick that turns this hard problem into a lower bound that we can actually compute.

Complete data. If we observed both $\mathbf{x}$ and $\mathbf{z}$, the objective would be

$$\log p(\mathbf{x}, \mathbf{z} \mid \theta) = \log p(\mathbf{x} \mid \mathbf{z}, \theta) + \log p(\mathbf{z} \mid \theta).$$

This follows from the product rule, $p(\mathbf{x}, \mathbf{z}) = p(\mathbf{x} \mid \mathbf{z})\, p(\mathbf{z})$, followed by $\log(ab) = \log a + \log b$. There is no integral and no log of a sum, so it is usually easy to maximize. For a Gaussian mixture it reduces to weighted averages.

Incomplete data (the real situation). We observe only $\mathbf{x}$, so the correct objective is

$$\log p(\mathbf{x} \mid \theta) = \log \int p(\mathbf{x}, \mathbf{z} \mid \theta)\, d\mathbf{z}.$$

Now the log sits outside an integral over hidden causes, the same difficulty as the log of a sum on slide 25.

Jensen's inequality. The slide says this produces a **variational lower bound**, "because log is concave":

$$\log \mathbb{E}[q] \;\geq\; \mathbb{E}[\log q].$$

- q: here, any positive random quantity. (The letter q is reused later for distributions, for example in $\mathrm{KL}(p\|q)$ on slide 31.)
- $\mathbb{E}[\cdot]$: the average.
- *Concave* means the curve bends downward like a hill, so a straight line between two points on it lies below the curve.

What it says: "average first, then take the log" is at least as big as "take the log first, then average." A tiny example with $\log_2$: let q be 1 or 4, each half the time. Then $\mathbb{E}[q] = 2.5$ and $\log_2 2.5 \approx 1.32$, while $\mathbb{E}[\log_2 q] = (0 + 2)/2 = 1$. Indeed $1.32 \geq 1$.

The practical payoff is that the right-hand side moves the log *inside* the average, where it can act on the easy pieces, and it is guaranteed to sit below the true value. That gives a computable lower bound to push upward. The phrase "in the right measure" means the average has to be taken under a suitable distribution over $\mathbf{z}$. Which one is the content of the VAE lecture. The hand notes stress that the bound exists because the latent is missing, not because of any particular neural architecture.

Slide 27: A taxonomy you will reuse all semester

This slide is a big table that sorts generative models into six families: autoregressive, variational autoencoder, GAN, diffusion, energy-based, and normalizing flow. Each row also says which week of the course covers that family. For every family, the table answers three questions: can the model give an exact probability for a piece of data, how does it make new samples, and what role does a hidden variable play. Two definitions under the table explain the words *implicit* and *unnormalized*.

Family	Density $p(\mathbf{x})$	Sampling	Latent $\mathbf{z}$
Autoregressive (W2, W9)	exact, via chain rule	sequential, exact	none required
Variational autoencoder (W3)	ELBO only	ancestral, one shot	explicit, continuous
GAN (W4)	implicit, not computed	one forward pass	noise input
Diffusion (W5–6)	ELBO or score	iterative denoising	noise trajectory
Energy-based (W7)	unnormalized $\tilde{p}$	MCMC, costly	none or hidden units
Normalizing flow (W8)	exact, via Jacobian	one invertible pass	the base noise

Reading the columns.

- *Density:* can the model give a number $p(\mathbf{x})$ for a given $\mathbf{x}$? Autoregressive models and flows can do it exactly. A VAE only gives a lower bound (the ELBO). Diffusion gives a bound or a score. GANs do not compute one at all.
- *Sampling:* how a new $\mathbf{x}$ is produced. Autoregressive models build $\mathbf{x}$ one piece at a time. A VAE draws $\mathbf{z}$ and decodes in one go. A GAN needs one forward pass through the generator. Diffusion removes noise step by step. Energy-based models use MCMC (a slow, step-by-step random-walk style sampler, described on the slide only as "costly"). A flow uses one invertible map.
- *Latent:* whether there is a hidden $\mathbf{z}$. Autoregressive models do not require one. A VAE has an explicit continuous code. For GANs, flows, and diffusion, the "latent" is the noise that goes in.

Two definitions. The slide says *implicit* means the model defines a sampler $G(\mathbf{z})$ but not a number $p(\mathbf{x})$. It says *unnormalized* means we know

$$\tilde{p}(\mathbf{x}) = e^{-E(\mathbf{x})} \quad \text{only up to a constant} \quad Z = \int \tilde{p}.$$

- $E(\mathbf{x})$: an "energy" score. Low energy means likely.
- $\tilde{p}(\mathbf{x})$: an unnormalized plausibility. It ranks points correctly but does not add up to 1.
- $Z = \int \tilde{p}(\mathbf{x})\, d\mathbf{x}$: the total, called the partition function. It is the area under $\tilde{p}$.

What it says: to get a real probability you would divide by Z, as in $p = \tilde{p}/Z$ (hand notes). But computing Z means integrating over every possible $\mathbf{x}$, which is usually infeasible, so p itself is unavailable.

Hand notes. Their version of the table words a few cells differently: flows are "exact, change of variables," energy-based models are "known up to $1/Z$," and the latent for energy-based models is "optional." It is the same content.

Slide 28: How the families relate

This slide is a family tree that groups generative models by what you are able to compute. At the very top is the general idea, a probabilistic generative model $p_\theta(\mathbf{x})$ (a tan box). It splits into three blue boxes: *tractable likelihood*, *bound or score*, and *implicit sampler*. Under each blue box are the specific model families that belong to it: AR (like PixelCNN and GPT) and Flow (like RealNVP and Glow) under the first, VAE and Diffusion under the second, and GAN under the third. In the rendered slide the bottom row of boxes overlaps, so parts of the AR and Flow labels are hidden behind their neighbors. The example names above are taken from the slide's text layer. Energy-based models are in the table on slide 27 but are not drawn in this tree.

What it says. The families are grouped by the *training signal*: what you are able to compute and optimize. Some can compute the likelihood exactly. Some can only compute a bound or a score. Some define only a sampler and compute no density. The grouping is **not** about image quality.

The caption adds a warning: a model can move between boxes. For example, a VAE with an autoregressive decoder has both a latent and a chain-rule likelihood term. So the tree is a rough map, not a set of walls.

Slide 29: What we will *not* treat as the same thing

This slide lists four pairs of ideas that sound alike but are actually different, so you should not mix them up. Each bullet has the form "A is not the same as B." The pairs are: an autoencoder versus a generative model, a GAN discriminator versus a plain classifier, sampling versus adding noise to a training image, and likelihood versus how good a picture looks.

1. **Autoencoder $\neq$ generative model.** A vanilla autoencoder maps $\mathbf{x} \to \mathbf{z} \to \hat{\mathbf{x}}$. It squeezes the input to a code and rebuilds it. But it does not define a prior $p(\mathbf{z})$ or a valid density, so there is no principled way to draw new data from it. A *variational* autoencoder adds both.
2. **Discriminator $\neq$ classifier of real data only.** In a GAN, the discriminator is trained adversarially against a moving generator. Its job keeps changing as the generator improves.
3. **Sampling $\neq$ "add Gaussian noise to a training image."** A sample is a draw from the model distribution. The hand notes call noising a training image data augmentation, not sampling from p_θ.
4. **Likelihood $\neq$ perceptual quality.** A model can have high likelihood and blurry samples (many VAEs), or sharp samples and no likelihood (GANs). Scoring well on one says little about the other.

Slide 30: Directed graphical model, in one slide

This slide explains directed graphical models, which are drawings where arrows show which variables depend on which. It gives one general rule for turning such a drawing into a formula, and then shows two example drawings. The left drawing has a hidden z with arrows pointing down to x_1, x_2, and x_3. The right drawing is a chain $x_1 \to x_2 \to x_3$, with an extra curved arrow that jumps from x_1 straight to x_3.

- **Left diagram (latent-variable joint):** a reddish circle z with three arrows down to x_1, x_2, x_3. Label: $p(z)\prod_d p(x_d \mid z)$.
- **Right diagram (fully autoregressive joint):** three circles in a row, $x_1 \to x_2 \to x_3$, plus a curved arrow jumping from x_1 directly to x_3. Label: $p(x_1)\,p(x_2 \mid x_1)\,p(x_3 \mid x_1, x_2)$.

The general rule. A directed graphical model (a Bayesian network) is a DAG, a *directed acyclic graph*: dots joined by arrows, with no loops. Each node v has a distribution that depends only on its parents. The joint is

$$p(\mathbf{v}) = \prod_v p\big(v \mid \mathrm{pa}(v)\big).$$

- $\mathbf{v}$: all the variables in the graph together.
- $\mathrm{pa}(v)$: the parents of node v, the nodes with an arrow pointing into v.
- $\prod_v$: multiply over every node.

What it says: to get the joint probability, multiply one factor per node, where each factor is "the chance of this node given its parents." A node with no parents just contributes its own plain probability.

Applying it to the two graphs.

- *Left:* z has no parents, so it contributes $p(z)$. Each x_d has the single parent z, so it contributes $p(x_d \mid z)$. The product is $p(z)\prod_d p(x_d \mid z)$.
- *Right:* x_1 has no parents, giving $p(x_1)$. x_2 has parent x_1, giving $p(x_2 \mid x_1)$. x_3 has parents x_1 and x_2 (that is what the curved arrow adds), giving $p(x_3 \mid x_1, x_2)$.

Why this matters. The slide says Week 2 builds the right-hand model with neural networks. There is no latent, and the likelihood is just a product of explicit conditionals, so it is exact. The left-hand model is easy to sample from but needs an integral over z to get $p(\mathbf{x})$. Both graphs define a legal joint. They simply make different quantities easy (hand notes).

Slide 31: Notation cheat sheet

This slide is a cheat sheet that lists the symbols we will use all semester. Each row gives a symbol and what the slide says it means. The table below adds a third column that says the same thing in plain English.

Symbol	Slide's meaning	Plain English
$\mathbf{x} \in \mathbb{R}^D$ or $\{0, \dots, 255\}^D$	one observation (image, sequence, ...)	a list of D numbers: real numbers, or whole numbers from 0 to 255 such as pixel values
$\mathcal{D} = \{\mathbf{x}^{(n)}\}_{n=1}^N$	training set; superscript (n) indexes points	the collection of all N examples we train on
y	observed label, when we have one	the tag, such as cat or dog
$\mathbf{z}$	latent variable, never in $\mathcal{D}$	the hidden cause; never in the data
θ	all trainable parameters	every knob the model can turn during training
p_{data}	unknown process that produced $\mathcal{D}$	nature's recipe, which we cannot see
$p_\theta(\mathbf{x})$	model density or mass function	our model's probability for $\mathbf{x}$
$p_\theta(\mathbf{x}, \mathbf{z})$	joint of observation and latent	chance of cause and observation together
$\ell(\theta) = \sum_n \log p_\theta(\mathbf{x}^{(n)})$	log-likelihood	the score we maximize in training
$\mathrm{KL}(p\|q) = \mathbb{E}_p[\log p - \log q]$	Kullback–Leibler divergence	how far q is from p

Two things to watch.

- The letter D is used twice. The script $\mathcal{D}$ is the dataset, and the plain D in $\mathbb{R}^D$ is the number of coordinates in one observation. The hand notes list them side by side as "$\mathbf{x}, \mathcal{D}, N, D$."
- The hand notes add one more symbol, $r_k(\mathbf{x})$: the posterior probability that mixture component k explains $\mathbf{x}$ (slide 24). They also repeat the rule: superscript (n) indexes data points, and subscripts index coordinates or mixture components. Do not swap them in derivations.

Slide 32: Mini-quiz (think, then check)

This slide is a short quiz of five questions so you can test yourself, with the answers printed at the bottom. The questions ask about density values bigger than 1, mixing up likelihood and posterior, why the hidden-variable integral is hard, whether a classifier can make new images, and why the coordinates in a mixture model depend on each other. Each question and answer is explained below.

1. "A density value $p(\mathbf{x}) = 4.2$ is illegal. True or false?" False. A density is a height, not a probability, and heights can exceed 1 as long as the total area is 1 (slide 7). What *cannot* exceed 1 is the probability of a region.

2. "You maximize $p(\theta \mid \mathcal{D})$ and call it maximum likelihood. What did you forget?" The likelihood is $p(\mathcal{D} \mid \theta)$, with the data on the left of the bar and the parameters on the right. $p(\theta \mid \mathcal{D})$ is the *posterior* over the parameters, and getting it requires a prior on θ. Maximizing it is called MAP (maximum a posteriori) or Bayesian estimation. It is a different method, not a mislabeled version of

the same one.

3. "Write $p(\mathbf{x})$ for a model with continuous latent z and state why a neural decoder makes this hard."

$$p(\mathbf{x}) = \int p(\mathbf{x} \mid \mathbf{z})\, p(\mathbf{z})\, d\mathbf{z}.$$

The decoder $p(\mathbf{x} \mid \mathbf{z})$ is a nonlinear neural network in $\mathbf{z}$, so the function being integrated is too tangled to integrate by a formula. The slide says the integral "does not factor through a nonlinear net." The hand notes put it as there being no closed-form antiderivative. Both mean there is no neat formula for the answer (slide 21).

4. "A classifier outputs $p(y \mid \mathbf{x})$. Can you sample a new image from it? What extra object would you need?" No. It reads images and gives labels, as in the top row of slide 16. To generate images you would also need $p(\mathbf{x})$, or $p(\mathbf{x} \mid y)$ together with $p(y)$, which is the generative path of slide 15.

5. "In a GMM, why is $p(x_1, x_2) \neq p(x_1)\, p(x_2)$ even if $p(x_1, x_2 \mid z) = p(x_1 \mid z)\, p(x_2 \mid z)$?" The two coordinates share the hidden cluster z. Writing out the marginal for a discrete z (hand notes),

$$p(x_1, x_2) = \sum_z p(x_1 \mid z)\, p(x_2 \mid z)\, p(z).$$

Each term is a product, but a *sum* of products usually does not factor into a product of sums. Summing z out breaks the product. Slide 23's umbrella example shows this with numbers.

Slide 33: What to carry into Week 2

This slide wraps up Week 1. It gives five short summary points about what you should remember, plus a teal box called "Before Thursday" that tells you what to read and practice before the next lecture.

The five bullets, in plain English.

1. Generation means a distribution $p_\theta(\mathbf{x})$, not only a function that outputs a prediction.
2. Maximum likelihood is the same as minimizing $\mathrm{KL}(p_{\text{data}} \| p_\theta)$, and it needs us to be able to compute $\log p_\theta(\mathbf{x})$ (slide 12).
3. Latent variables explain why coordinates depend on each other, but the marginal likelihood has to integrate them out (slides 20–23).
4. Discriminative models answer "which label?" Generative models answer "what does data look like?" (slides 15–18).
5. Getting an exact $\log p(\mathbf{x})$ is a design choice. Next week we get it from the chain rule, at the price of sampling one piece at a time.

Before Thursday. Read Tomczak Ch. 1 and Bishop's section on latent variables. Be ready to expand $p(x_1, x_2, x_3)$ with the chain rule in two different orders.

The hand notes state the general chain rule for Week 2:

$$p(x_1, \ldots, x_D) = \prod_{i=1}^{D} p(x_i \mid x_1, \ldots, x_{i-1}).$$

- i: the position of a coordinate in the chosen order.
- $p(x_i \mid x_1, \ldots, x_{i-1})$: the chance of coordinate i given all the coordinates before it. For $i = 1$ there is nothing before it, so the factor is just $p(x_1)$.

What it says: you can build the probability of a whole object by going one piece at a time, each time asking "how likely is this piece given everything so far." This is just the product rule from slide 8 applied over and over.

For three variables, the order x_1, x_2, x_3 gives the expansion on slide 30:

$$p(x_1, x_2, x_3) = p(x_1)\, p(x_2 \mid x_1)\, p(x_3 \mid x_1, x_2).$$

Any other order is equally valid. For example, the order x_3, x_1, x_2 gives

$$p(x_1, x_2, x_3) = p(x_3)\, p(x_1 \mid x_3)\, p(x_2 \mid x_1, x_3).$$

Both equal the same joint, because the product rule holds no matter which variable you peel off first. The two products differ only in how the work is split into conditionals (hand notes).

Slide 34: References for this lecture

This slide lists five books and papers you can read to learn more about the topics in this lecture.

- Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006, Ch. 8–9 (graphical models, mixtures, EM).
- Murphy, *Probabilistic Machine Learning: An Introduction*, MIT Press, 2022, Ch. 3.
- Tomczak, *Deep Generative Modeling*, Springer, 2022, Ch. 1.
- Goodfellow, Bengio, and Courville, *Deep Learning*, MIT Press, 2016, §5.5 and Ch. 20 (marked "preview").
- Ng and Jordan, "On discriminative vs. generative classifiers," NeurIPS, 2001. This is the paper behind the comparison on slide 17.

As noted on slide 2, the readings slide gives Bishop Ch. 8–10 while this slide gives Ch. 8–9.

Appendix A: Hand-notes items not tied to one slide

Mistakes that lose points. The hand notes list six. In plain English, with where each is explained:

1. Calling $p(\theta \mid \mathcal{D})$ the likelihood. The likelihood is $p(\mathcal{D} \mid \theta)$ (slides 11 and 32).
2. Treating a density value above 1 as illegal. A *probability* above 1 is illegal; a density height is not (slides 7 and 32).
3. Writing $p(\mathbf{x}) = \int p(\mathbf{x} \mid \mathbf{z})\, d\mathbf{z}$ and dropping the prior $p(\mathbf{z})$ (slide 20).
4. Claiming a classifier $p(y \mid \mathbf{x})$ can generate images. It cannot unless $p(\mathbf{x})$ or $p(\mathbf{x} \mid y)$ is also available (slides 15, 16, 32).
5. Concluding $x_1 \perp x_2$ from $x_1 \perp x_2 \mid z$. Independence given z does not mean independence overall (slides 23 and 32).
6. Swapping log and $\sum_k$ in a mixture. $\log \sum_k \pi_k \mathcal{N}_k$ is not $\sum_k \log(\pi_k \mathcal{N}_k)$ (slide 25).

EM formulas and the exam. Know the responsibility $r_k(\mathbf{x})$ and the statement of the E-step and M-step. Do not memorize the update formulas unless a problem asks you to derive a specific M-step (slide 25).

Worked check. The hand notes finish with worked answers to four of the mini-quiz questions (slide 32): the density above 1, the likelihood versus posterior mix-up, the hard integral, and the dependent coordinates in a mixture. They agree with the slide.

Appendix B: Where Homework 1 draws its ideas from

Homework 1 covers Lecture 1 and the Week 1 hand notes. It is due at the start of the Week 2 lecture, is worth 100 points, and asks for derivations with the steps shown. It also asks you to do the write-up, algebra, and numerical work yourself and not to look up worked solutions, so the table below only points to where each topic is explained. It contains no solutions.

Problem (points)	Slides	Notes
1. Joint, marginal, conditional (16)	8, 9, 15, 16, 17	Product rule; marginal as a sum over labels; what a discriminative model can and cannot do
2. Bernoulli and Gaussian MLE (20)	11, 12, 13	Slide 13 works the Bernoulli case. The Gaussian case is not worked in the lecture or hand notes, so use the readings
3. Likelihood, KL, and what the words mean (18)	7, 11, 12, 32	Probability versus likelihood; the KL identity on slide 12; densities versus probabilities
4. A two-component mixture (24)	23, 24, 32	Mixture density; responsibility; why marginal dependence appears
5. Latent variables and the course map (22)	20, 21, 27, 28	Marginal likelihood, joint, posterior; why the integral is hard; the family table